

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

**Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей**

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 13

Дисциплина: Архитектура компьютера

«Программирование в командном процессоре ОС UNIX. Ветвления и циклы»

Студент: Пириев Керем

Группа: НПИбд 02–25

Москва, 2026 г.

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Выполнение работы

1. Создание рабочего каталога

Создан каталог ~/lab13 для размещения всех скриптов:

bash

mkdir -p ~/lab13

cd ~/lab13

```
kerem@fedora:~/lab13$ nano search.sh
kerem@fedora:~/lab13$ chmod +x search.sh
kerem@fedora:~/lab13$ echo "Hello world" > test.txt
kerem@fedora:~/lab13$ echo "hello again" >> test.txt
kerem@fedora:~/lab13$ ./search.sh -i test.txt -p world
Hello world
kerem@fedora:~/lab13$ ./search.sh -i test.txt -p hello -C
Hello world
hello again
kerem@fedora:~/lab13$
```

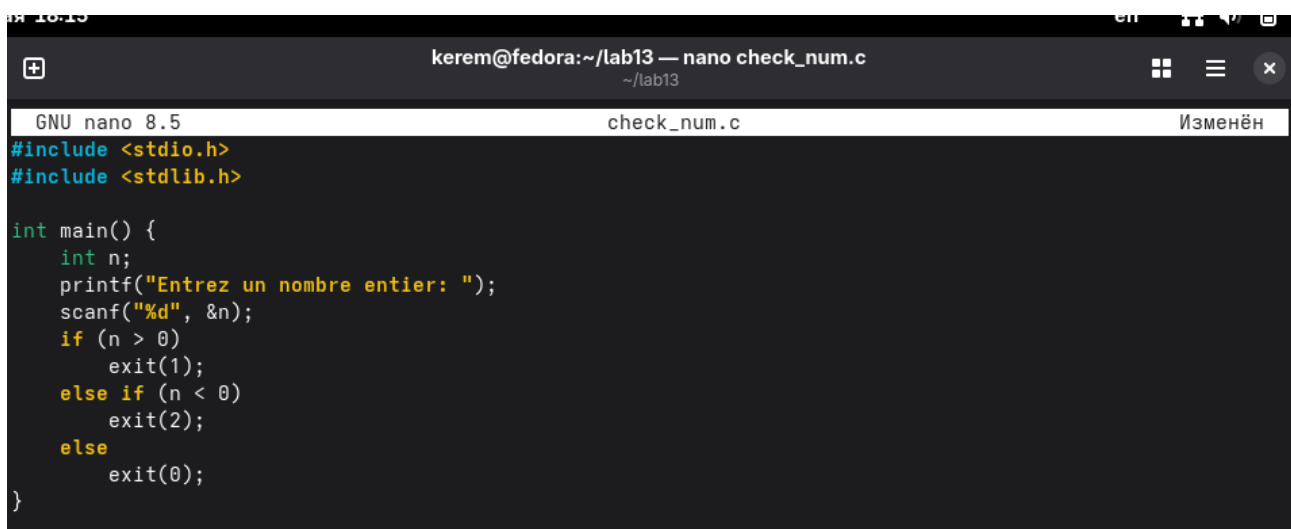
echo "Hello world" > test.txt

echo "hello again" >> test.txt

chmod +x search.sh

./search.sh -i test.txt -p world

./search.sh -i test.txt -p hello -C



```
kerem@fedora:~/lab13 — nano check_num.c
~/lab13
GNU nano 8.5                                check_num.c                                Изменён
#include <stdio.h>
#include <stdlib.h>

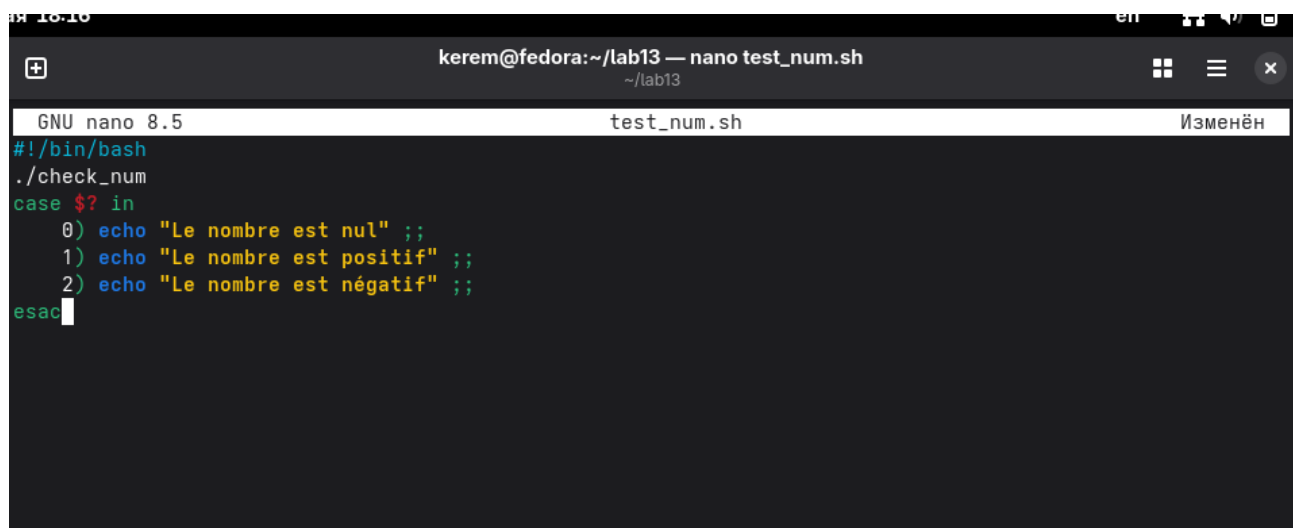
int main() {
    int n;
    printf("Entrez un nombre entier: ");
    scanf("%d", &n);
    if (n > 0)
        exit(1);
    else if (n < 0)
        exit(2);
    else
        exit(0);
}
```

3. Программа на C + скрипт bash (анализ кода возврата)

3.1. Программа на C (check_num

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Entrez un nombre entier: ");
    scanf("%d", &n);
    if (n > 0)
        exit(1);
    else if (n < 0)
        exit(2);
    else
        exit(0);
}
```



```
kerem@fedora:~/lab13 — nano test_num.sh
~/.lab13
GNU nano 8.5 test_num.sh Изменён
#!/bin/bash
./check_num
case $? in
  0) echo "Le nombre est nul" ;;
  1) echo "Le nombre est positif" ;;
  2) echo "Le nombre est négatif" ;;
esac
```

3.2. Скрипт bash (test_num.sh)

Создан файл test_num.sh:

bash

```
#!/bin/bash
./check_num
case $? in
  0) echo "Le nombre est nul" ;;
  1) echo "Le nombre est positif" ;;
  2) echo "Le nombre est négatif" ;;
esac
```

```

kerem@fedora:~/lab13$ nano check_num.c
kerem@fedora:~/lab13$ gcc check_num.c -o check_num
kerem@fedora:~/lab13$ nano check_num.c
kerem@fedora:~/lab13$ nano test_num.sh
kerem@fedora:~/lab13$ chmod +x test_num.sh
kerem@fedora:~/lab13$ ./test_num.sh
Entrez un nombre entier: 58
Le nombre est positif
kerem@fedora:~/lab13$

```

Компиляция и запуск:

bash

```

gcc check_num.c -o check_num
chmod +x test_num.sh
./test_num.sh

```

4. Скрипт для создания / удаления временных файлов (files.sh)

Создан файл files.sh:

bash

```

#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <N> (créer N fichiers .tmp)"
    echo "Pour supprimer: $0 clean"
    exit 1
fi

if [ "$1" = "clean" ]; then
    rm -f [0-9]*.tmp
    echo "Fichiers .tmp supprimés."
    exit 0
fi

N=$1

```

```
for ((i=1; i<=N; i++)); do
    touch "$i.tmp"
done
echo "$N fichiers créés :"
ls -l [0-9]*.tmp 2>/dev/null || echo "Aucun fichier .tmp trouvé."
```

```
kerem@fedora:~/lab13$ nano files.sh
kerem@fedora:~/lab13$ chmod +x files.sh
kerem@fedora:~/lab13$ ./files.sh 5
5 fichiers créés :
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 1.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 2.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 3.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 4.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 5.tmp
kerem@fedora:~/lab13$ ./files.sh clean
Fichiers .tmp supprimés.
kerem@fedora:~/lab13$
```

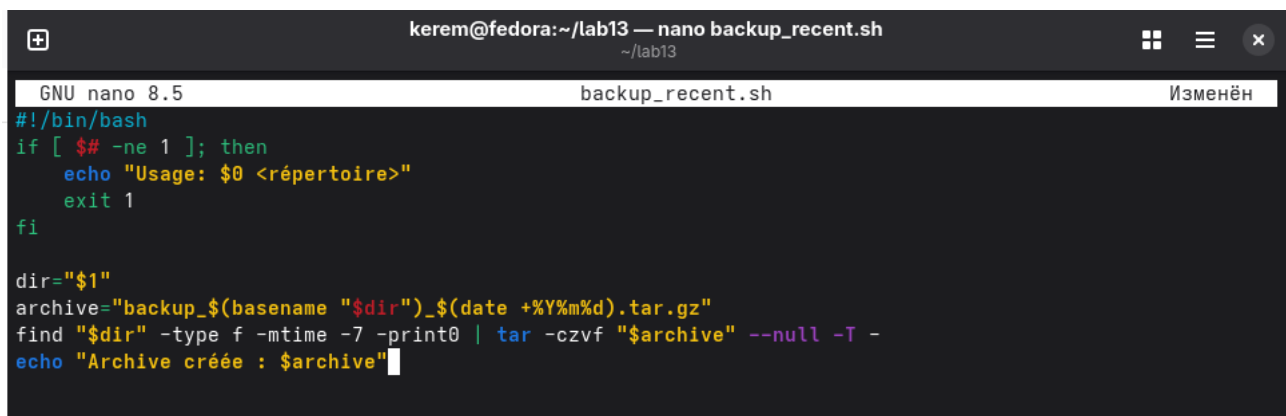
5. Скрипт для архивации недавно изменённых файлов (backup_recent.sh)

Создан файл backup_recent.sh:

bash

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <répertoire>"
    exit 1
fi

dir="$1"
archive="backup_$(basename "$dir")_$(date +%Y%m%d).tar.gz"
find "$dir" -type f -mtime -7 -print0 | tar -czvf "$archive" --null -T -
echo "Archive créée : $archive"
```



```
kerem@fedora:~/lab13 — nano backup_recent.sh
GNU nano 8.5 backup_recent.sh
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <répertoire>"
    exit 1
fi

dir="$1"
archive="backup_$(basename "$dir")_$(date +%Y%m%d).tar.gz"
find "$dir" -type f -mtime -7 -print0 | tar -czvf "$archive" --null -T -
echo "Archive créée : $archive"
```

```
mkdir -p testdir
touch testdir/oldfile
touch -d "2 days ago" testdir/recentfile
chmod +x backup_recent.sh
./backup_recent.sh testdir
ls -l backup_*.tar.gz
```

```
kerem@fedora:~/lab13$ chmod +x backup_recent.sh
kerem@fedora:~/lab13$ mkdir -p testdir
kerem@fedora:~/lab13$ touch testdir/oldfile
kerem@fedora:~/lab13$ touch -d "2 days ago" testdir/recentfile
kerem@fedora:~/lab13$ ./backup_recent.sh testdir
testdir/oldfile
testdir/rece
testdir/recentfile
Archive créée : backup_testdir_20260531.tar.gz
kerem@fedora:~/lab13$ ls -l backup_*.tar.gz
-rw-r--r--. 1 kerem kerem 165 мая 31 18:33 backup_testdir_20260531.tar.gz
kerem@fedora:~/lab13$
```

Выводы

В ходе работы были написаны и отлажены четыре скрипта:

1. **search.sh** – скрипт поиска строк в файле с поддержкой опций командной строки (`getopts`): выбор входного и выходного файла, шаблон поиска, учёт регистра (`-C`), нумерация строк (`-n`).
2. **test_num.sh + check_num.c** – демонстрация анализа кода возврата программы на C в `bash`-скрипте с использованием `case`.
3. **files.sh** – скрипт для массового создания временных файлов и их удаления.
4. **backup_recent.sh** – скрипт для архивации файлов, изменённых за последние 7 дней.

Получены практические навыки использования:

- цикла `for`;
 - условного оператора `if`;
 - оператора выбора `case`;
 - разбора аргументов командной строки (`getopts`);
 - работы с кодами возврата (`$?`);
 - подстановки команд и работы с `find`, `tar`.
-

Ответы на контрольные вопросы

1. Какие символические имена (метасимволы) существуют в оболочке `bash`?

Метасимвол	Назначение
<code>*</code>	Любая строка (0 и более символов)
<code>?</code>	Один любой символ
<code>[]</code>	Один символ из указанного диапазона или набора
<code>{ }</code>	Генерация списка строк (например, <code>{1..5}</code>)
<code> </code>	Конвейер (передача вывода одной команды на ввод другой)
<code>&</code>	Запуск команды в фоновом режиме
<code>;</code>	Разделитель команд
<code>()</code>	Группировка команд в подчинённой оболочке
<code><</code>	Перенаправление ввода
<code>></code>	Перенаправление вывода (с перезаписью)
<code>>></code>	Перенаправление вывода (с добавлением)
<code>\$</code>	Подстановка значения переменной

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (`*`, `?`, `[ ]`) используются для **шаблонов имён файлов (globbing)**. Оболочка автоматически заменяет шаблон на список имён файлов, соответствующих ему.

Пример: `*.txt` соответствует всем файлам с расширением `.txt` в текущем каталоге.

3. Какие операторы управления действиями вы знаете?

- **Условные:** `if/then/else/elif/fi`, `case/esac`
- **Циклы:** `for`, `while`, `until`
- **Управление циклом:** `break`, `continue`
- **Завершение скрипта:** `exit`

4. Какие операторы используются для прерывания цикла?

- `break` – **полный выход** из цикла.
- `continue` – **переход к следующей итерации** цикла (пропуск оставшейся части текущей итерации).

5. Для чего нужны команды `false` и `true`?

- `true` – всегда возвращает код возврата **0** (успех).
- `false` – всегда возвращает ненулевой код возврата (обычно 1, ошибка).

Используются для организации бесконечных циклов (`while true`), логических проверок или как «заглушки».

6. Что означает строка `if test -f man\$$/\$i.\$$`, встреченная в командном файле?

(С учётом возможного экранирования)

Эта строка проверяет, существует ли файл с именем, составленным из:

- каталог `man\$$` (где `\$` — экранированный символ `$`, то есть буквальный знак доллара, а не переменная);
- далее `/` ;
- значение переменной `$i` ;
- символ `.` ;
- значение переменной `$$`.

Обратный слеш `\` перед `$` экранирует специальное значение знака доллара, чтобы оболочка не интерпретировала `$$` как переменную (если это требуется по логике скрипта).

7. Объясните различие между конструкциями `while` и `until`.

- **`while`** — цикл выполняется **пока условие истинно** (т.е. пока код возврата команды-условия равен 0).

bash

```
while условие; do
    команды
done
```

`until` — цикл выполняется **пока условие ложно** (т.е. повторяется, пока код возврата команды-условия не станет 0). Другими словами, `until` выполняется **до тех пор, пока условие не станет истинным**.

bash

```
until условие; do
    команды
done
```

Пример различия:

- `while [ "$i" -lt 10 ]` — повторять, пока `i` меньше 10.
- `until [ "$i" -ge 10 ]` — повторять, пока `i` **не станет** больше или равно 10 (то есть **пока i меньше 10**).

